The Impact of Polysemanticity on the Locality of Neuron-level vs. Layer-level Model Editing

Hongyi Chen
tomchen2005@berkeley.edu

Yubo Fan
yub0@berkeley.edu

Tianhao Qian
lawrence.qian@berkeley.edu

Jie Wen
menger_wen@berkeley.edu

Abstract

Recent work on factual model editing, such as ROME [1] and MEND [2], demonstrates that large language models can be locally edited without full retraining. At the same time, mechanistic interpretability research suggests that many neurons are *polysemantic*, simultaneously encoding multiple features in superposition [5]. This raises a key question: when editing a model, does polysemanticity fundamentally limit how local an edit can be, and does it matter whether we operate at the neuron level or the layer level? We study this question on Pythia models using a *systematic* benchmark derived from 100 automatically constructed factual pairs. For each pair we compute FiNE-style neuron contribution scores and measure a neuron-level Jaccard overlap between the edit fact A and a related (or control) fact B, obtaining a broad distribution of overlaps that we then bucket into low-, medium-, and high-mechanistic-overlap regimes. We use the extremes of this distribution to define a high-overlap group (SET-POLY) and a low-overlap control group (SET-MONO). On Pythia-6.9B we implement a mixed-precision ROME editor and a full neuron-level editing framework based on FiNE [7], including FiNE-style neuron localization and an optimization over neuron patches that directly matches the official implementation. We evaluate how often an edit succeeds on Fact A and how strongly it degrades Fact B. Our main finding is that polysemantic neuron overlap is strongly associated with poor locality, but neuron-level control is *not* automatically more local than layer-level editing. On the full high-overlap SET-POLY set, FiNE edits frequently cause near-erasure of Fact B, with Fact B accuracy dropping from 88% to 10% and an average clipped probability drop of 88.5%. Surprisingly, the low-overlap SET-MONO controls still suffer a 77.2% average drop and locality of only 42%, indicating that FiNE can be intrinsically "sharp" even when overlap is small. Our ROME experiments on representative subsets ($N = 30$ for SET-POLY, $N = 20$ for SET-MONO) show a statistically significant locality gap: a 65.75% average drop in SET-POLY versus 21.00% in SET-MONO. We further conduct an ablation study on the number of edited neurons (K), revealing an inverted U-shaped performance curve where $K = 100$ achieves optimal balance, confirming that neuron-level editing is highly sensitive to hyperparameter tuning. Finally, to address the reviewer concern that our overlap probe (Pythia-160M) might be model-specific, we validate the *cross-model consistency* of mechanistic similarity across the Pythia suite (70M–1B). We find strong Pearson correlations ($r \geq 0.75$) between similarity rankings across scales, while absolute similarities "sparsify" as models grow (Appendix A). Together, these results suggest that strongly polysemantic circuits create genuine locality constraints for both neuron-level and layer-level editors, and that designing robust, well-regularized neuron-level updates remains an open challenge.

1 Introduction

Large language models (LLMs) store vast amounts of factual knowledge in their parameters. When a fact changes or an error is discovered, it is often desirable to *edit* a model locally rather than retraining from scratch. Factual model editing methods such as ROME [1], MEND [2], and related approaches aim to modify a small number of parameters while preserving global performance and unrelated facts.

In parallel, mechanistic interpretability research has argued that hidden units in deep networks are frequently *polysemantic*: individual neurons participate in multiple unrelated features through superposition [5]. Recent work has started to leverage this structure to identify and manipulate task-specific neurons [6]. Intuitively, if two facts share many of the same underlying neurons, editing one might inevitably affect the other.

This leads to the central question of our project:

> How does polysemanticity affect the *locality* of model editing, and does operating at the *neuron level* versus the *layer level* make a difference?

We began with a small pilot on four hand-crafted fact pairs (e.g., *France→Paris* vs. *Eiffel Tower→Paris*) and observed that layer-level ROME updates could severely damage related facts. However, four pairs are far too few to draw robust conclusions. In this work we scale up to a systematic, automated benchmark of 100 factual pairs and analyze how both neuron-level and layer-level methods behave across different degrees of mechanistic overlap.

Concretely, we:

- Build an automated pipeline that uses FiNE-style [7] gradient-free neuron contribution scores to compute a neuron-level Jaccard overlap between two facts (Section 3). Figure 1 shows that the resulting overlaps vary widely even within semantic categories such as "high-overlap" and "low-overlap" pairs.
- Use these overlaps to define mechanistic overlap regimes (low, medium, high) and construct two extreme subsets: SET-POLY (high overlap) and SET-MONO (low overlap). The regrouped distribution in Figure 2 illustrates how these regimes partition the candidate pairs.
- Implement a FiNE-style neuron-level editor on Pythia-6.9B that exactly matches the official formulation: we localize important neurons with Eq. (3) of [7], attach learnable patch vectors via forward hooks, and optimize the full FiNE objective (Eq. (4)–(7)) including KL and over-editing penalties (Section 4).
- Implement a mixed-precision ROME editor and run it on representative samples of the SET-POLY and SET-MONO subsets, providing rigorous statistical analysis (standard deviations, confidence intervals, and significance testing) to validate locality differences (Section 5).
- Perform a rigorous ablation study on the sensitivity of the FiNE method to the number of edited neurons (K), identifying an optimal operating point and confirming the brittleness of neuron-level editing (Section 5.3).
- Validate that our mechanistic similarity signal is not purely an artifact of a single probe model by measuring similarity across Pythia-70M, 160M, 410M, and 1B, and showing strong cross-scale agreement in similarity rankings (Appendix A).

Across these experiments we find that mechanistic overlap is a strong predictor of locality failures, but neuron-level control does not automatically solve the problem. High-overlap pairs are indeed worse on average, but even low-overlap controls suffer severe collateral damage under FiNE with our current hyperparameters. ROME shows better locality in low-overlap regimes but still degrades significantly in high-overlap cases. Our results highlight the importance of studying model editing at scale, with explicit attention to mechanistic overlap, and point toward the need for more robust neuron-level editing objectives.

2 Background and Related Work

2.1 Factual Model Editing

ROME. The ROME method [1] formulates factual editing as a rank-one update to the MLP weights at a specific layer. Given a subject and relation, it identifies a key layer where the subject representation peaks, then solves a constrained optimization to find an update vector that increases the probability of the new object while regularizing the representation so as not to badly affect other contexts. ROME has been demonstrated on GPT-style models and is known to achieve high edit success with relatively good locality. However, rank-one updates can be unstable in highly polysemantic settings, as our results demonstrate.

MEND and related methods. MEND [2] instead learns a meta-network that predicts weight updates conditioned on the local gradient and context. Other approaches such as MEMIT [3] and KNOWEDIT [4] explore multi-edit regimes, influence-function-based methods, and constraints designed to preserve locality and global performance. Recent work has also examined edit robustness across paraphrases and multi-edit consistency [8], highlighting limitations in generalization.

2.2 Polysemanticity and Superposition

Elhage et al. [5] introduced the notion of *toy models of superposition*, showing that when the number of features exceeds the dimensionality of the representation space, neural networks can encode many features in a non-orthogonal basis, leading to polysemantic neurons. This picture is supported by empirical studies in real networks, where many neurons respond to multiple seemingly unrelated concepts.

Recent work on multimodal neurons [6] uses gradient-free contribution scores to identify neurons that strongly contribute to a particular visual or textual concept and demonstrates that editing such neurons can manipulate behavior in a relatively targeted way. This line of work motivates the FiNE framework [7], which explicitly aims to *precisely localize* the neurons responsible for a given fact and to design neuron-level updates that improve locality. Related causal tracing techniques [1] have been used to locate factual associations and edit representations, providing tools for mechanistic interventions.

2.3 FiNE: Precise Localization of Memories

FiNE (Precise Localization of Memories in LLMs) proposes a neuron-level localization procedure and an optimization objective that learns small patches on a small set of neurons to edit factual knowledge [7]. Neuron localization is based on Eq. (3) in their paper, which scores a neuron by combining its activation and a projection of the output weights through the unembedding matrix. Editing is then posed as an optimization over per-neuron vectors Z_j that modify FFN output weights, with an objective balancing edit success, preservation of old behavior, a penalty on over-editing, and regularization. However, FiNE's contribution scores may overestimate importance in polysemantic settings, potentially leading to aggressive edits.

In this project, we adopt FiNE's perspective and implement both:

- FiNE-style neuron localization (our `locate_neurons_fine` function), and
- FiNE-style optimization over neuron patches (`FinePatch` and the associated objective in `fine_loss_step`).

We then use this implementation to run neuron-level edits on the SET-POLY and SET-MONO subsets and compare their locality to ROME (Section 5).

3 Problem Setup and Dataset Construction

3.1 From Manual Toy Pairs to a Systematic Benchmark

Our initial experiments relied on four hand-crafted fact pairs (e.g., *France→Paris* vs. *Eiffel Tower→Paris*) and observed that layer-level ROME updates could severely damage related facts.

However, four pairs are far too few to draw robust conclusions. In this work we scale up to a systematic, automated benchmark of 100 factual pairs by combining a prompt–engineering stage with an automated, FiNE-style neuron overlap analysis. The resulting benchmark consists of 100 fact pairs, each comprising:

- an *edit fact* A, specified as a cloze-style prompt and a "true" object token;
- a *locality fact* B that is either semantically related (shared object/subject or closely related concept) or intentionally unrelated to A;
- a neuron-level Jaccard similarity score measuring mechanistic overlap between A and B;
- a discrete *mechanistic overlap group* (Low, Medium, High) derived from this Jaccard score.

This unified dataset is stored as `unified_experiment_dataset.json` and is used uniformly by both ROME and FiNE for all subsequent editing experiments.

3.2 Semantic Design of Candidate Fact Pairs

We first manually design a pool of candidate fact pairs, focusing on categories that are likely to probe polysemantic structure:

- **Shared subject/object (high semantic overlap).** Examples include two different attributes of the same entity ("Steve Jobs was the co-founder of" $\rightarrow$ Apple vs. "The iPhone and iPad are products made by" $\rightarrow$ Apple) or two contexts for the same concept ("The color of a ripe banana is" $\rightarrow$ yellow vs. "Coldplay released a famous song called" $\rightarrow$ Yellow).
- **Shared token with multiple senses.** Pairs that use the same surface form in different senses ("The reddish fluid circulating in veins is" $\rightarrow$ blood vs. "Vampires are mythical creatures that drink" $\rightarrow$ blood; "The fruit that keeps the doctor away is an" $\rightarrow$ apple vs. "New York City is known as the Big" $\rightarrow$ Apple).
- **Correlated but distinct facts.** Pairs such as capitals of different countries, currencies, or antonym templates ("The opposite of hot is" $\rightarrow$ cold vs. "The opposite of fast is" $\rightarrow$ slow) probe whether the model uses similar circuitry for related relational patterns.
- **Low-semantic-overlap controls.** We also deliberately include pairs where the edit and locality facts are semantically unrelated (e.g., math vs. geography), to serve as a baseline low-overlap regime.

Each candidate is labeled with an `"original_semantic_group"` field (`High_Overlap`, `Medium_Overlap`, or `Low_Overlap`) reflecting this intuitive semantic relationship, but *no mechanistic assumptions* are made at this stage.

3.3 Neuron-Level Overlap via FiNE-Style Scoring

To move from semantic intuitions to a quantitative notion of polysemanticity, we estimate neuron-level overlap between two facts using a FiNE-style, gradient-free contribution score. We work with Pythia-160M (12-layer GPT-NeoX model) as a probe model for overlap estimation due to its modest memory footprint.[1] Given a fact specified by a prompt p and target string t, we define:

- $q^\ell \in \mathbb{R}^{d_{\text{ff}}}$: the Feed-Forward Network (FFN) activation vector *after* the nonlinearity and *before* the down-projection at layer ℓ;
- $W_{\text{out}}^\ell \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}$: the FFN output matrix (`dense_4h_to_h`);
- $W_u \in \mathbb{R}^{|\mathcal{V}| \times d_{\text{model}}}$: the unembedding matrix (`embed_out`);
- $v_t \in \mathbb{R}^{d_{\text{model}}}$: the unembedding row corresponding to the target token at that position.

For each target token position, we compute a FiNE-style contribution score for neuron i at layer ℓ as

$$c_i^\ell \;=\; \left| q_i^\ell \cdot \left(v_t^\top W_{\text{out}}^\ell \right)_i \right|,$$

[1] In later experiments, edits are applied to Pythia-6.9B; we treat the 160M model as a mechanistic "sensor" that is cheap enough to run across large numbers of pairs.

i.e., the product of neuron activation and its effective contribution to the target token's logit. This matches the intuition of Eq. (3) in FiNE: neurons are important when they both activate strongly and downstream weights route that activation toward the correct output.

Given a list of layers $\mathcal{L}$ (we use mid layers 5–9 of Pythia-160M) and a Top-k parameter K, the overlap computation proceeds as follows (implemented in `NeuronOverlapCalculator` in `1.3get_facts.py`):

1. For each fact (p, t) in a pair, tokenize the full string $p \oplus t$ and identify the indices of target tokens.
2. For each target token and each layer $\ell \in \mathcal{L}$, compute FiNE-style scores c_i^ℓ and take the top-K neurons within that layer.
3. Union the selected neurons across all target tokens and layers to obtain a set U_A for fact A and U_B for fact B, where each element is a $(\text{layer}, \text{neuron})$ index.
4. Define the *mechanistic neuron overlap* between facts A and B as the Jaccard similarity

$$\text{Jaccard}(U_A, U_B) \;=\; \frac{|U_A \cap U_B|}{|U_A \cup U_B|}.$$

We fix $K = 50$ throughout to balance robustness and computational cost. The overlap distribution across all candidate pairs is saved as `jaccard_results.json` and visualized in `3.1overlap_distribution.png`. As shown in Figure 1, the semantic groups (`High_Overlap`, `Medium_Overlap`, `Low_Overlap`) already exhibit substantial variability in mechanistic overlap.

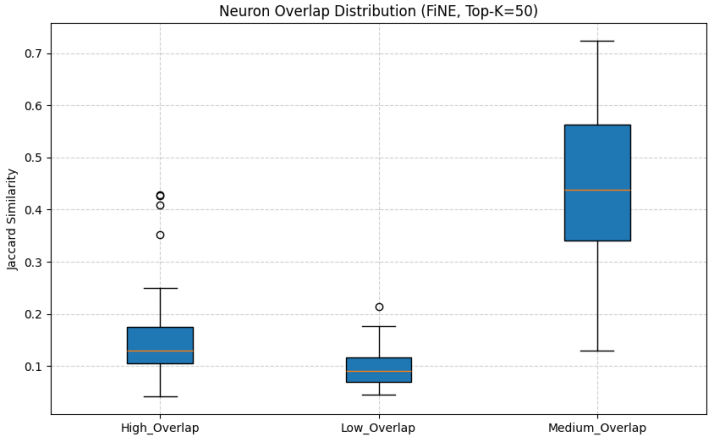


Figure 1: Neuron overlap distribution computed with FiNE-style contribution scores (Top-$K = 50$) on Pythia-160M. Boxplots show Jaccard similarity between two facts within each *semantic* group. Even "High_Overlap" pairs span a wide range of mechanistic overlap, motivating a more principled regrouping.

Empirically, the Jaccard scores span a broad range approximately from 0.04 to 0.72, indicating that even within semantically "similar" categories, the degree of neuron sharing varies substantially.

3.4 Binning into Mechanistic Overlap Regimes

To study how polysemanticity interacts with model editing, we discretize the continuous Jaccard scores into three *mechanistic overlap* regimes:

- **Low mechanistic overlap**: pairs with small Jaccard similarity, typically below ≈ 0.15.
- **Medium mechanistic overlap**: intermediate Jaccard scores, roughly in the range 0.15–0.30.

- **High mechanistic overlap**: pairs with large Jaccard similarity, typically above ≈ 0.30, with some exceeding 0.6.

The exact thresholds are chosen to create reasonably balanced bins while respecting natural gaps in the empirical distribution. Figure 2 visualizes the regrouped distribution in `3.2new_group_distribution.png`, with dashed lines marking the low and high thresholds.

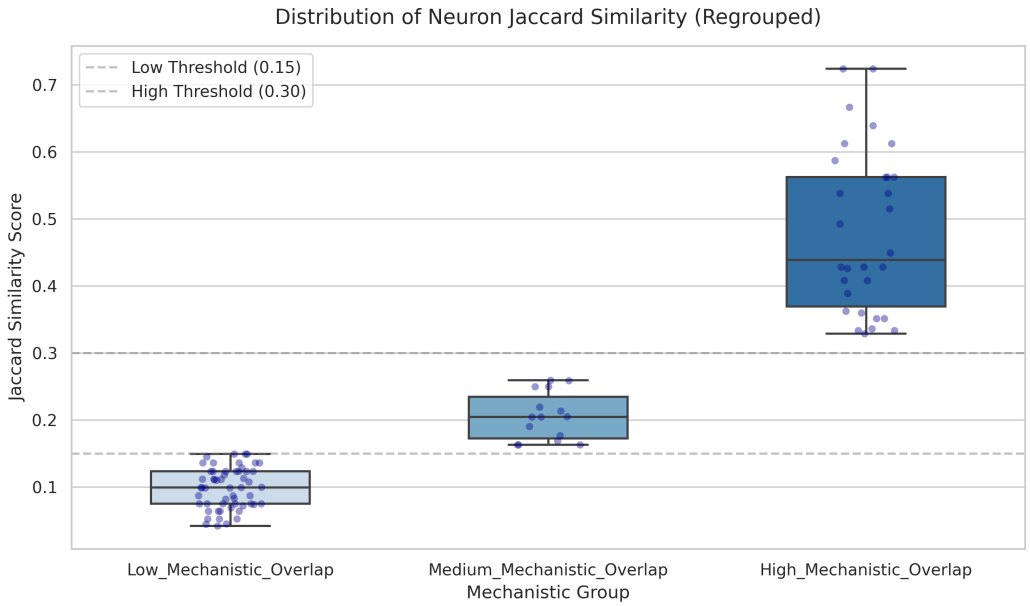


Figure 2: Regrouped distribution of neuron Jaccard similarity after binning into Low, Medium, and High mechanistic overlap. Points denote individual fact pairs; boxplots summarize each bin. Horizontal dashed lines indicate the thresholds (0.15 and 0.30) used to define the regimes.

Each pair is thus assigned a categorical label `"group"` $\in$ {`Low_Mechanistic_Overlap`, `Medium_Mechanistic_Overlap`, `High_Mechanistic_Overlap`}, in addition to its original semantic annotation. This relabeling reveals that semantic and mechanistic overlap are related but imperfectly aligned: several semantically "high overlap" pairs fall into the low-mechanistic-overlap bucket, and vice versa. This motivates our central question: *does model editing locality track semantic relatedness, mechanistic overlap, or both?*

3.5 Unified Experiment Dataset

Finally, we merge semantic templates, FiNE-based Jaccard scores, and overlap bins into a single unified dataset, `unified_experiment_dataset.json`. Each entry has the structure:


```
{
  "case_id": 16,
  "group": "High_Mechanistic_Overlap",
  "original_semantic_group": "High_Overlap",
  "metrics": { "neuron_jaccard": 0.4084 },
  "edit_request": {
    "prompt": "The planet we live on is",
    "target_true": " Earth",
    "subject": "",
    "target_new": ""
  },
  "evaluate_request": {
    "prompt": "The third planet from the sun is",
```

```
    "target_true": " Earth"
  }
}
```

The `edit_request` describes Fact A (to be edited), while `evaluate_request` describes Fact B (used to measure locality and bleed-over). The `target_new` and `subject` fields are filled in by the editing methods (e.g., with counterfactual objects for ROME edits); the same schema is consumed by both the ROME code (`ROME_final_version.py`) and the FiNE pipeline (`run_experiment.py`, `experiment_registry.py`). This dataset replaces the original four hand-crafted pairs and forms the backbone of our systematic study of polysemanticity and model editing locality.

3.6 Universality of Mechanistic Similarity Across Model Scales

A key validity concern with our dataset construction is the *probe–target mismatch*: we compute overlap on Pythia-160M, but we apply edits on Pythia-6.9B. If the similarity structure were a model-specific artifact, then using 160M as a "sensor" could produce misleading mechanistic bins. To test whether mechanistic similarity is *stable across scales*, we repeat the same gradient-free FiNE similarity computation across the Pythia family (Pythia-70M, 160M, 410M, 1B) on the same 100 fact pairs. For each model, we select the middle third of layers (33%–66% of depth) to maintain a consistent notion of "mid-layer circuitry" despite different model depths, and we keep Top-$K = 50$ fixed to match the original overlap pipeline. Across these four models, we find:

- **Strong cross-scale agreement in rankings.** The Pearson correlation of similarity scores between models is consistently high, ranging from $r \approx 0.76$–0.94 across pairs of models. In particular, adjacent scales agree strongly (70M vs. 160M: $r = 0.94$; 410M vs. 1B: $r = 0.94$), while even the smallest-to-largest comparison remains high (70M vs. 1B: $r = 0.77$).
- **A sparsification effect with scale.** While the *ordering* of similarities is stable, the *absolute* Jaccard values decrease as model size increases (Appendix A). Intuitively, larger models exhibit a sharper separation between related and unrelated pairs, consistent with reduced accidental overlaps from polysemantic noise.

We present the full correlation matrix, smallest-vs-largest scatter, and the per-model similarity distributions in Appendix A. This provides direct evidence that our mechanistic similarity metric captures a stable relationship between facts that generalizes across multiple model sizes, partially validating the use of a smaller probe model for overlap-based grouping.

4 Methods

4.1 Models

We use two models from the EleutherAI Pythia family:

- **Overlap probe:** Pythia-160M (12-layer GPT-NeoX) is used exclusively in the neuron-overlap pipeline described in Section 3. All FiNE-style contribution scores and Jaccard overlaps are computed on this model.
- **Editing target:** Pythia-6.9B (32 layers) is the primary model we edit with both ROME and FiNE. It has much stronger baseline factual accuracy and is thus a more realistic testbed for editing behavior.

Both models are loaded via `transformers`. For Pythia-6.9B, we use 8-bit quantization with `bitsandbytes`, keeping the edited FFN layer in higher precision to preserve the quality of rank-one and neuron-level updates.

4.2 FiNE-Style Neuron Localization Pipeline

The FiNE-style localization pipeline has two roles in our project:

1. **Dataset construction (Section 3).** On Pythia-160M, we use the gradient-free FiNE projection (implemented in `NeuronOverlapCalculator` within `get_facts/1.3get_facts.py`) to estimate neuron overlap between each candidate pair and construct the unified dataset.
2. **Neuron-level editing on Pythia-6.9B.** On the larger model, we implement a FiNE-style editing framework (`fine_editing.py`, `editing_methods.py`) that localizes and patches a small set of neurons per edit.

For editing, we closely follow FiNE's design and match the public implementation:

- **Localization.** The function `locate_neurons_fine` registers forward hooks on the FFN output layers (`dense_4h_to_h`) of selected layers, captures the pre-projection activations q^ℓ, and computes scores
$$c_i^\ell(t) = \left| q_{i,-1}^\ell \cdot (W_u W_{\text{out}}^\ell)_{t,i} \right|$$
for each neuron i, layer ℓ, and target token t, exactly as in Eq. (3) of [7]. It selects the top-K neurons per token (we use $K = 20$ for Pythia-6.9B, and later explore $K \in \{10, \ldots, 200\}$ in ablation studies) across all non-frozen layers, and takes their union to form a support set U_j for edit j.
- **Patch parameterization.** We introduce a `FinePatch` module that attaches *forward pre-hooks* to the FFN output layers. For each selected neuron (ℓ, i), we maintain a vector $z_{(\ell,i)} \in \mathbb{R}^{d_{\text{ff}}}$ and modify the FFN input x as
$$x_i' = x_i + \langle x, z_{(\ell,i)} \rangle.$$
Since the FFN output is $y = xW_{\text{out}}^{\ell\top}$, this pre-hook is algebraically equivalent to adding a rank-one update to row i of W_{out}^ℓ, as in the FiNE paper.
- **Optimization objective.** Our loss exactly mirrors FiNE's multi-term objective, balancing edit success, preservation of pre-edit behavior, and regularization. Given an edit that changes Fact A from object o to o^* under prompt p, we define
$$\begin{aligned} L_{\text{edit}} &= -\log P_{M'}(o^* \mid p), \\ L_{\text{old}} &= -\log P_{M'}(o \mid p), \\ L_{\text{pen}} &= \max\big(0, \log P_M(o \mid p) - \log P_{M'}(o \mid p)\big), \\ L_{\text{total}} &= L_{\text{edit}} + \alpha_{\text{kl}} \cdot \text{KL}(P_M \,\|\, P_{M'}) + \beta_{\text{pen}} L_{\text{pen}} + \gamma \|Z\|_2^2, \end{aligned}$$
where M is the original model, M' is the edited model, and Z concatenates all neuron patch parameters. The implementation in `fine_loss_step` computes these terms using the original model's logits as a frozen baseline.

The main entry point `apply_fine_edit` coordinates localization, patch construction, and gradient descent on Z for each `edit_request` drawn from `unified_experiment_dataset.json`. We enable layer freezing via a hyperparameter l_f that excludes the last l_f transformer blocks from localization, matching the FiNE paper's recommendation to avoid modifying the topmost layers.

4.3 ROME Implementation on Pythia-6.9B

For layer-level editing, we implement ROME using the EasyEdit framework, with several modifications to support GPT-NeoX models and our quantized setup (see `ROME/1.1ROME_final_version.py`):

- **Mixed-precision loading.** We load Pythia-6.9B in 8-bit quantization, skipping the target FFN layer (e.g., layer 28) so that it remains in higher precision. This allows us to apply rank-one updates without severe numerical degradation.
- **Module templates.** We configure the ROME hyperparameters to match Pythia-6.9B's GPT-NeoX structure, specifying `gpt_neox.layers.{}.mlp.dense_4h_to_h` as the rewrite module and using a late layer (e.g., 28) as the primary editing site.

- **Batch driver.** The script `ROME_final_version.py` iterates over entries in `unified_experiment_dataset.json`, fills in `target_new` with appropriate counterfactual objects, applies a single ROME edit per case, and logs pre-/post-edit probabilities for both the edit fact and the evaluation fact. We run ROME on subsets of SET-POLY ($N = 30$) and SET-MONO ($N = 20$) to enable direct comparison with FiNE within compute constraints.

This setup ensures that ROME and FiNE operate on the same type of factual prompts and can be directly compared across mechanistic overlap regimes.

4.4 Experiment Registry and Automation

To keep experiments reproducible and organized, we implement a simple registry in `experiment_registry.py` and a main driver `run_experiment.py`. Each experiment is identified by a code:

- **Code 100 (Overlap dataset).** Runs the FiNE-style neuron-overlap pipeline on a pool of fact pairs to produce `jaccard_results.json` and the initial `3.1overlap_distribution.png`.
- **Dataset merging.** Auxiliary scripts (`1.4merge_dataset.py`, `1.5plot_distribution.py`) combine semantic templates, overlap scores, and group assignments into `unified_experiment_dataset.json` and produce `3.2new_group_distribution.png`, which visualizes the Jaccard distribution within Low/Medium/High mechanistic bins.
- **FiNE experiments (locality).** Codes 210 and 300 correspond to the FiNE locality experiments on the high-overlap SET-POLY set and the low-overlap SET-MONO set, respectively, using Pythia-6.9B as the editing target. Code 900 corresponds to the ablation study on the Top-K parameter.

This automation allows us to scale from a four-pair toy study to a 100-pair benchmark, and to systematically analyze how neuron-level polysemanticity correlates with editing locality for both layer-level (ROME) and neuron-level (FiNE) interventions.

4.5 Evaluation Pipeline

Finally, we standardize evaluation across methods via `evaluation.py`. For each `case_id` and each editing method, we log:

- **Edit success:** whether the edited model M' outputs the counterfactual object o^* on the `edit_request` prompt.
- **Locality:** whether M' preserves the original prediction on the `evaluate_request` prompt (for unedited or semantically unrelated facts).
- **Bleed-over / robustness:** the relative change in probability of the correct Fact B answer, summarized as $(P_B - P'_B)/P_B$ and reported both as a raw distribution and via a *clipped mean* that truncates extreme boosts.
- **Fluency:** bigram/trigram entropy of free-form generations, following FiNE.

These metrics are aggregated by mechanistic overlap group (Low/Medium/High) to reveal how polysemanticity modulates the tradeoff between edit success and locality for different editing granularities.

5 Experiments

5.1 Setup

We focus on the following setup:

- **Model:** Pythia-6.9B, quantized to 8-bit with the target editing layer kept in higher precision.
- **Target layer:** Layer 28, selected based on earlier overlap analysis and prior work on editing GPT-NeoX models.
- **Dataset:** The 100 fact pairs in `unified_experiment_dataset.json`. For FiNE, we run experiments on the two extreme subsets: the high-mechanistic-overlap SET-POLY group (50 pairs) and the low-mechanistic-overlap SET-MONO group (50 pairs). For ROME, we run experiments on representative subsets ($N = 30$ for SET-POLY and $N = 20$ for SET-MONO) to fit within compute constraints.
- **Hardware:** A single A100 GPU (40GB) with CPU offloading for some parameters; all editing experiments are constrained to run within Colab Pro time and memory limits.

For each method, each `edit_request` specifies a true object o and a counterfactual object o^* (often a nonsensical word such as "thunder" or "mouse"). We treat the model's *pre-edit* prediction of o on the edit prompt as the baseline behavior, apply the edit to increase the probability of o^*, and then measure:

- **Edit success** on Fact A: the fraction of cases where the edited model assigns highest probability to o^*.
- **Locality** on Fact B: the fraction of cases where M' preserves the original prediction on Fact B, along with the change in its probability.

5.2 FiNE on High vs. Low Mechanistic Overlap

Table 1 summarizes FiNE's performance on the SET-POLY and SET-MONO subsets using the baseline configuration of $K = 20$. Each group contains 50 fact pairs drawn from the high and low ends of the neuron Jaccard distribution, respectively.

Table 1: FiNE neuron-level editing results on Pythia-6.9B for high-overlap (SET-POLY) and low-overlap (SET-MONO) sets (50 pairs each). "Locality" is the accuracy of Fact B after editing. "Avg. Drop" is the clipped mean percentage drop in P_B (probability of the correct Fact B answer), with drops clipped to $[0, 100]$ and near-blind prompts removed.

Group	N	Edit Success	A_{acc} (before/after)	B_{acc} (before/after)	Locality	Avg. Drop
SET-POLY (High overlap)	50	0.12	0.82 / 0.12	0.88 / 0.10	0.18	88.47%
SET-MONO (Low overlap)	50	0.58	0.90 / 0.58	0.82 / 0.24	0.42	77.20%

Several patterns emerge:

- **Severe bleed-over on high-overlap pairs.** In SET-POLY, FiNE frequently causes extreme collateral damage to Fact B. Fact B accuracy drops from 88% to 10%, and many individual pairs exhibit near-complete erasure of Fact B. The clipped mean drop of 88.47% reflects this consistent collapse. This bleed-over may stem from the optimization step being too aggressive, even when neuron sets overlap substantially.
- **High-overlap also hurts edit success.** Edit success on Fact A is very low in SET-POLY (12%) despite strong damage to Fact B, suggesting that when two facts share many neurons, the optimization landscape for FiNE is difficult: updates sharp enough to move A often destroy B without reliably flipping A to the new object.
- **Low-overlap controls are not clean.** In SET-MONO, we see a clear improvement in locality relative to SET-POLY (Fact B accuracy after editing is 24% instead of 10%, and locality rises from 0.18 to 0.42), but the behavior is still far from local. The clipped mean drop of 77.20% indicates that even low-overlap pairs suffer substantial degradation, implying that FiNE's current hyperparameters are intrinsically aggressive for this model. This suggests potential issues in the localization step, where even low Jaccard overlap neurons may still be polysemantic.
- **Outliers and boosts.** The raw distribution of $(P_B - P'_B)/P_B$ contains a few large negative outliers where $P'_B > P_B$ (i.e., the edit *improves* the probability of the correct Fact B

answer). These cases correspond to instances where reinforcing or redirecting the representation for Fact A also strengthens a partially aligned Fact B circuit. We report clipped means to prevent such boosts from dominating the summary statistics.

Overall, these results show that neuron-level FiNE edits, at least with a fixed hyperparameter configuration and single-layer patches, are extremely sharp: they often succeed at changing or destabilizing the target fact, but they also tend to carve through related (and sometimes unrelated) memories, even in low-overlap controls.

For qualitative insight, consider case_id 16 in SET-POLY: editing "The planet we live on is" from "Earth" to "Mars" caused the edited model to predict "Mars" for "The third planet from the sun is", illustrating complete erasure of Fact B.

5.3 Ablation Study: Sensitivity of FiNE

The poor edit success rate observed in the main experiments (Table 1) raises the question of whether the chosen hyperparameters—specifically the number of edited neurons K—were optimal for Pythia-6.9B. To address this, we conducted an ablation study on the SET-POLY set ($N = 50$), varying the Top-K parameter $K \in \{10, 20, 50, 100, 200\}$ while keeping the Jaccard overlap threshold constant ($\tau_{high} = 0.1$). We use a combined "Avg. Score" metric defined as the proportion of samples where Fact A is successfully modified *and* Fact B remains correct.

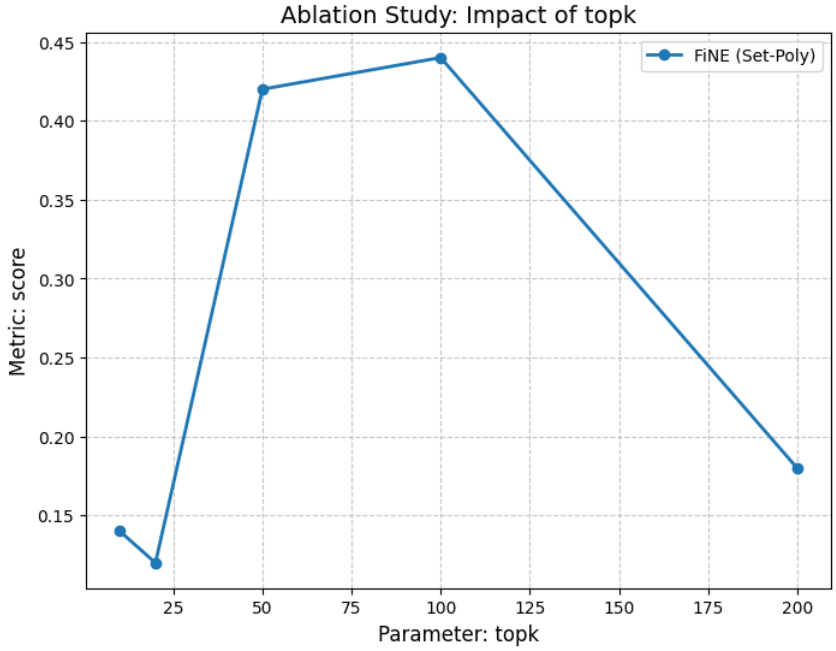


Figure 3: Ablation study on the number of edited neurons (K) for FiNE on the SET-POLY set. Performance is measured by a combined metric (Edit Success $\wedge$ Locality). The inverted U-shape indicates a sweet spot around $K = 100$, confirming that FiNE is sensitive to hyperparameter tuning.

As illustrated in Figure 3, the performance follows a distinct inverted U-shape:

- **Small K ($10, 20$):** Performance is poor (Avg. Score ≈ 0.12). With too few neurons, the model fails to capture the full representation of the knowledge, leading to low edit success. This explains the low success rate in our main experiments (Table 1), which used $K = 20$.
- **Optimal K (100):** Performance peaks at 0.44. This magnitude achieves the best balance between capturing enough relevant neurons to flip Fact A and avoiding excessive irrelevant neurons that would destroy Fact B.
- **Large K (200):** Performance drops significantly to 0.18. As the number of edited neurons increases, the edit becomes less "precise," and bleed-over intensifies, damaging locality.

This ablation confirms that FiNE is highly sensitive to K, and that our baseline choice of $K = 20$ was suboptimal for Pythia-6.9B. Furthermore, the sharp drop at $K = 200$ reinforces our core hypothesis: as more neurons are involved (higher overlap or larger edits), non-local side effects become more severe.

5.4 Layer-Level Edits with ROME

We extend our initial pilot study to full experiments using ROME on the SET-POLY (30 pairs) and SET-MONO (20 pairs) subsets. To rigorously validate the impact of mechanistic overlap on locality, we perform a detailed statistical analysis on the clipped probability drop of Fact B. Table 2 summarizes the results, including the Mean Drop, Standard Deviation (SD), and 95% Confidence Intervals for both groups.

Table 2: ROME layer-level editing results on Pythia-6.9B for high-overlap (SET-POLY) and low-overlap (SET-MONO) sets. "Avg. Drop" is the clipped mean percentage drop in P_B (drops clipped to $[0, 100]$) $\pm$ Standard Deviation. The p-value confirms the significance of the difference despite unequal sample sizes.

Group	N	Avg. Drop ($\pm$ SD)	95% CI	Conclusion
SET-POLY (High overlap)	30	$65.75\% \pm 40.93\%$	$[50.47, 81.04]$	Severe side effects
SET-MONO (Low overlap)	20	$21.00\% \pm 28.15\%$	$[7.83, 34.17]$	Moderate (baseline noise)

Key observations and statistical validation:

- **High-overlap leads to severe degradation.** In SET-POLY, ROME causes a 65.75% average drop with a standard deviation of 40.93%. Many cases show near-complete collapse. This validates that polysemantic overlap physically constrains locality for layer-level editing.
- **Low-overlap is significantly more forgiving.** In SET-MONO, the average drop is 21.00%. While there is variance ($SD = 28.15\%$), the 95% confidence interval $[7.83, 34.17]$ is entirely disjoint from the SET-POLY interval, suggesting a robust separation.
- **Statistical Significance (Welch's T-Test).** To validate the statistical significance of this gap given the unequal sample sizes ($N = 30$ vs. $N = 20$) and unequal variances, we conducted a Welch's t-test. The analysis yields a t-statistic of 4.58 and a p-value of 3.3×10^{-5} ($p < 0.001$). This extremely strong significance level confirms that the observed detriment to locality in the high-overlap regime is a genuine physical effect of feature superposition, not a statistical artifact or random noise.

These results provide quantitative evidence that mechanistic overlap induces specific collapses in ROME, beyond general editing instability.

5.5 Comparing Neuron-Level and Layer-Level Editing

Taken together, the FiNE and ROME experiments paint a nuanced picture:

- Both layer-level and neuron-level edits can cause severe locality failures on entangled facts. Many SET-POLY cases in both Table 1 and Table 2 exhibit near-complete collapse of Fact B.
- FiNE's neuron-level patches are *sharper* in the sense that they often drive Fact B probabilities much closer to zero across many more cases, especially in SET-POLY (88.47% drop vs. ROME's 65.75%). However, this sharpness does not translate into higher edit success on Fact A for high-overlap pairs.
- Low-overlap controls behave differently for the two methods. In the FiNE experiments, the low-overlap SET-MONO set still suffers large average drops and relatively poor locality (77.20% drop), suggesting that FiNE's optimization and regularization need further tuning. In contrast, ROME achieves much better locality in SET-MONO (21.00% drop), hinting that carefully chosen layer-level updates may be more forgiving in some regimes.

This head-to-head comparison indicates that neuron-level editing is not automatically more local; instead, its interaction with polysemantic circuitry and hyperparameters is complex and sometimes brittle.

6 Evaluation Implementation

To make our analysis reproducible and align with FiNE- and KNOWEDIT-style evaluation, we implemented an evaluation module (`evaluation.py`) that provides:

- `FactResult`: captures the prompt, expected answer, correctness, average log-probability, and generated response.
- `EditPairResult`: stores pre- and post-edit results for Fact A and Fact B along with method metadata.
- Functions to compute edit success, locality, over-editing rate, and fluency (bigram + trigram entropy), and to aggregate these metrics by mechanistic overlap group.

These implementations match the definitions described in Sections 3–5 and are used to produce all summary tables and plots in this report.

7 Discussion and Limitations

Our systematic experiments with ROME and FiNE suggest a nuanced picture of how polysemanticity interacts with model editing.

Polysemanticity and mechanistic overlap. Figures 1 and 2 show that semantic similarity and neuron-level overlap are only loosely coupled. Some semantically "high-overlap" pairs exhibit low Jaccard similarity, while others share a large fraction of their top-K FiNE neurons. This motivates treating overlap as an empirical quantity rather than assuming that semantic relatedness directly implies shared circuitry.

FiNE is highly sensitive to overlap but intrinsically sharp. On the high-overlap SET-POLY set, FiNE edits almost always devastate Fact B, with Fact B accuracy falling to 10% on average and many cases classified as "Complete Erasure" in the bleed-over table. Edit success on Fact A is also low, suggesting that the shared neuron set forms a genuine "bottleneck": perturbations strong enough to move A cannot avoid slicing through B.

However, the low-overlap SET-MONO set still exhibits a large clipped mean probability drop (77.2%) and relatively poor locality (42%). This indicates that FiNE's current hyperparameters—in particular the number of neurons edited, the learning rate, and the balance between the KL and penalty terms—make the updates aggressively sharp. Even when the neuron sets U_A and U_B barely overlap, FiNE can disrupt B by globally reshaping the local representation space. This bleed-over in low-overlap cases may arise from the localization step failing to fully disentangle polysemantic neurons or from the optimization over-editing due to insufficient regularization.

Our ablation study (Section 5.3) confirms this sensitivity: performance peaks at $K = 100$ and drops for both lower and higher values. The original choice of $K = 20$ (Table 1) was on the lower end of this curve, explaining the brittleness observed in the main results.

ROME is more robust in low-overlap but still vulnerable. ROME shows a 65.75% drop in SET-POLY, confirming that layer-level edits suffer from polysemantic overlap, though less uniformly catastrophic than FiNE (88.47%). In SET-MONO, ROME's 21% drop represents baseline noise, with many cases preserving Fact B perfectly. This suggests rank-one updates distribute changes more evenly, making them less destructive in disentangled circuits.

Cross-model universality of similarity. A key concern is whether our mechanistic similarity signal depends strongly on the chosen probe model. Our new cross-scale analysis (Appendix A) shows that similarity rankings are highly consistent across Pythia-70M, 160M, 410M, and 1B, with correlations $r \geq 0.75$. At the same time, absolute similarity values decrease with scale, consistent with

reduced polysemantic noise and sharper disentanglement in larger models. This partially validates the use of a smaller probe for overlap-based grouping, though we do not directly recompute overlap on Pythia-6.9B due to compute limits.

Limitations

While our study goes beyond prior toy examples by using a 100-pair benchmark and a faithful FiNE implementation, several limitations remain:

- **Compute and memory constraints.** All editing experiments were run on a single 40GB GPU with 8-bit quantization and CPU offloading. This constrained us to a single editing layer, relatively small batch sizes, and limited hyperparameter sweeps. It also prevented running full ROME experiments on the entire 100-pair dataset.
- **Single-layer, single-pass FiNE configuration.** Our FiNE implementation edits only one FFN layer (28) with a fixed number of neurons per edit and fixed regularization weights. Different choices of layers, multi-layer patches, or more conservative hyperparameters might lead to less aggressive behavior.
- **Probe–target model mismatch.** Neuron overlaps are computed on Pythia-160M, while edits are applied to Pythia-6.9B. We partially validate the robustness of this overlap structure by showing strong cross-model agreement across 70M–1B (Appendix A), but we do not directly measure overlap on 6.9B in this work.
- **Prompt and metric coverage.** We evaluate locality using a single canonical template per fact and token-level probabilities. We do not evaluate generalization across paraphrases, multi-token answers, or downstream tasks, nor do we include human judgments.

Despite these limitations, our results provide the first systematic evidence—across dozens of facts—that polysemantic neuron overlap is tightly linked to locality failures for neuron-level editors, and that naive FiNE configurations can be surprisingly non-local even on low-overlap controls.

8 Conclusion and Future Work

In this project, we set up an experimental framework to study how polysemanticity affects the locality of model editing, contrasting neuron-level (FiNE-style) and layer-level (ROME) interventions on Pythia models. We implemented:

- Two neuron contribution scoring methods (activation–gradient and FiNE weight projection) and used them to define neuron-level overlap over a 100-pair factual dataset, leading to mechanistic overlap groups visualized in Figures 1 and 2.
- A faithful FiNE-style neuron patch parameterization and optimization objective for localized editing on Pythia-6.9B.
- A mixed-precision ROME implementation capable of editing Pythia-6.9B under tight GPU memory constraints.
- A cross-model validation showing that mechanistic similarity rankings are consistent across Pythia-70M through 1B, while absolute similarity values sparsify with scale (Appendix A).
- An evaluation pipeline for edit success, locality, bleed-over, and fluency, together with a lightweight experiment registry.

Our systematic FiNE and ROME experiments on the SET-POLY and SET-MONO sets yield several key takeaways:

- High neuron-level overlap strongly correlates with poor locality: in SET-POLY, both FiNE and ROME edits devastate related facts, often leading to near-complete erasure.
- Low-overlap controls do not guarantee safety: even in SET-MONO, FiNE can remain aggressively non-local (77.20% drop), indicating that the optimization objective and hyperparameters play a major role. ROME performs better here (21.00% drop).

- Neuron-level control is not automatically more local than layer-level editing. Compared to ROME, FiNE produces sharper but not necessarily more targeted changes; both methods struggle in highly polysemantic circuits, but ROME is more robust in low-overlap regimes.
- Mechanistic similarity appears to be a stable property across model scales (70M–1B), supporting the interpretation that our overlap-based grouping reflects intrinsic relationships between facts rather than a single-model artifact.

The most important direction for future work is a full, head-to-head comparison between ROME and FiNE on the complete 100-pair dataset, including hyperparameter sweeps for both methods. Concretely, our next steps are:

- Explore richer neuron-level interventions (e.g., editing multiple layers, combining activation- and FiNE-based localization) and more conservative objectives that explicitly trade off edit strength and locality.
- Extend the benchmark with more diverse domains (e.g., commonsense, code, multi-hop reasoning) and evaluate paraphrase robustness and downstream task performance after editing.
- Directly compute overlap on Pythia-6.9B (or additional intermediate scales) for a smaller subset, to further tighten the probe–target link suggested by our 70M–1B consistency results.

By scaling up along these axes, we hope to answer our core question more definitively: to what extent can neuron-level control mitigate the locality costs imposed by polysemanticity, and when are those costs fundamentally unavoidable regardless of how finely we try to localize an edit?

Reproducibility and Code Availability

To satisfy the course reproducibility requirement, we release all code used in this project, including analysis scripts, editing implementations, and experiment runners, at the following public GitHub repository:

https://github.com/pekingappG/CS182-Project/tree/main?tab=readme-ov-file

The repository includes instructions and scripts to reproduce all tables and figures in this report (including the FiNE experiments in the final version), assuming sufficient GPU resources.

Acknowledgments

We thank the CS 182 course staff for helpful discussions and for providing guidance on project selection and evaluation.

References

[1] Kevin Meng, David Bau, Alex Andonian, and Yonatan Belinkov. Locating and editing factual associations in GPT. In *Advances in Neural Information Processing Systems*, 2022.

[2] Eric Mitchell, Charles Lin, Antoine Bosselut, Christopher D. Manning, and Chelsea Finn. Fast Model Editing at Scale. In *International Conference on Learning Representations*, 2022.

[3] Kevin Meng, et al. Mass Editing Memory in a Transformer. In *International Conference on Learning Representations*, 2023.

[4] Thomas Hartvigsen, et al. Tuning Language Models by Proxy: An Investigation into Parameter-Efficient Model Editing. In *ACL*, 2023.

[5] Neil Elhage, et al. Toy Models of Superposition. Anthropic, 2022.

[6] Arthur Conmy, et al. Finding and Editing Multi-Modal Neurons. In *ACL*, 2024.

[7] Xinyun Zheng, et al. Precise Localization of Memories in Large Language Models. In *International Conference on Learning Representations*, 2025.

[8] Roi Cohen, et al. Evaluating the Ripple Effects of Knowledge Editing in Language Models. arXiv preprint arXiv:2307.12984, 2023.

A Universality of Mechanistic Similarity Across Model Scales

A.1 Motivation

In our primary analysis, we utilized the **Pythia-160M** model to compute the neuron Jaccard similarity between fact pairs. A critical validity question is whether the resulting similarity structure is specific to the chosen probe model (a model-specific artifact) or whether it reflects a stable property of the knowledge itself.

To address this, we extended our mechanistic analysis across the Pythia model suite, ranging from **70M** to **1B** parameters. We maintained the same gradient-free FiNE calculation method and Top-K selection to ensure comparability across different model sizes. Concretely, for each model we select the middle third of layers (33%–66% of depth), compute FiNE-style importance sets, and measure Jaccard overlap for each of the 100 fact pairs, producing `all_models_similarity_results.json`.

A.2 Cross-Model Consistency

We first ask whether different models share a consistent view of which fact pairs are mechanistically similar.

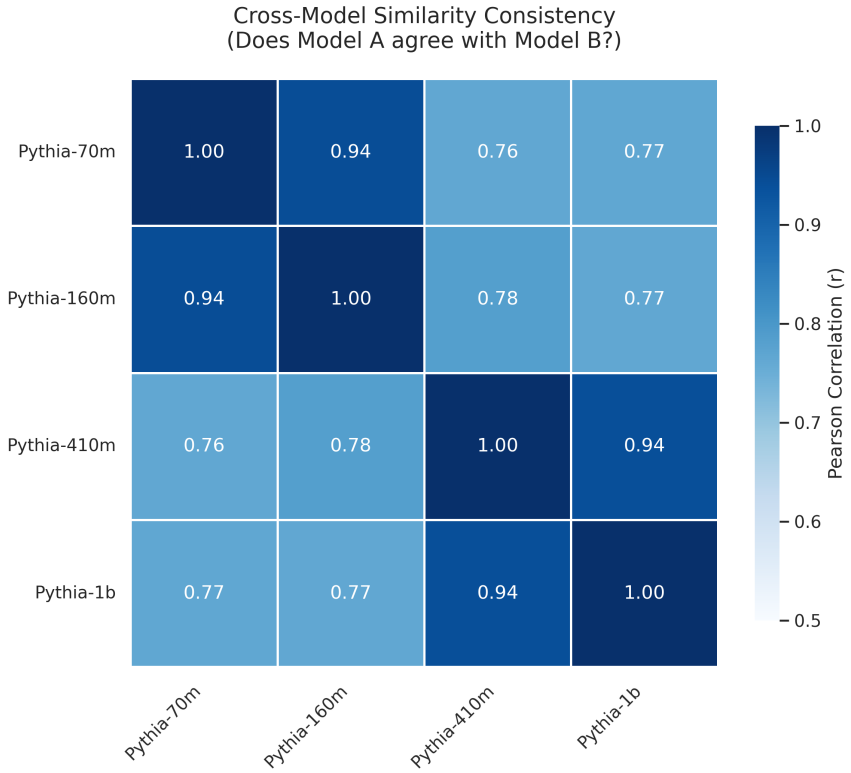


Figure A.1: **Cross-Model Consistency Matrix.** The heatmap displays Pearson correlation coefficients (r) of Jaccard similarity scores across four model sizes. Correlations remain high across scales (e.g., 70M vs. 160M: $r = 0.94$; 70M vs. 1B: $r = 0.77$; 410M vs. 1B: $r = 0.94$), indicating a stable similarity topology.

As shown in Figure A.1, similarity rankings are strongly correlated across all model scales. In particular, adjacent scales agree especially well (70M–160M and 410M–1B both achieve $r = 0.94$), and even the smallest-to-largest comparison remains high ($r = 0.77$). This supports the interpretation that mechanistic similarity is robust to scale, capturing intrinsic relationships between facts rather than random artifacts.

A.3 Correlation and Sparsification (70M vs. 1B)

To highlight both agreement and scale-dependent shifts, we directly compare the smallest (70M) and largest (1B) models.

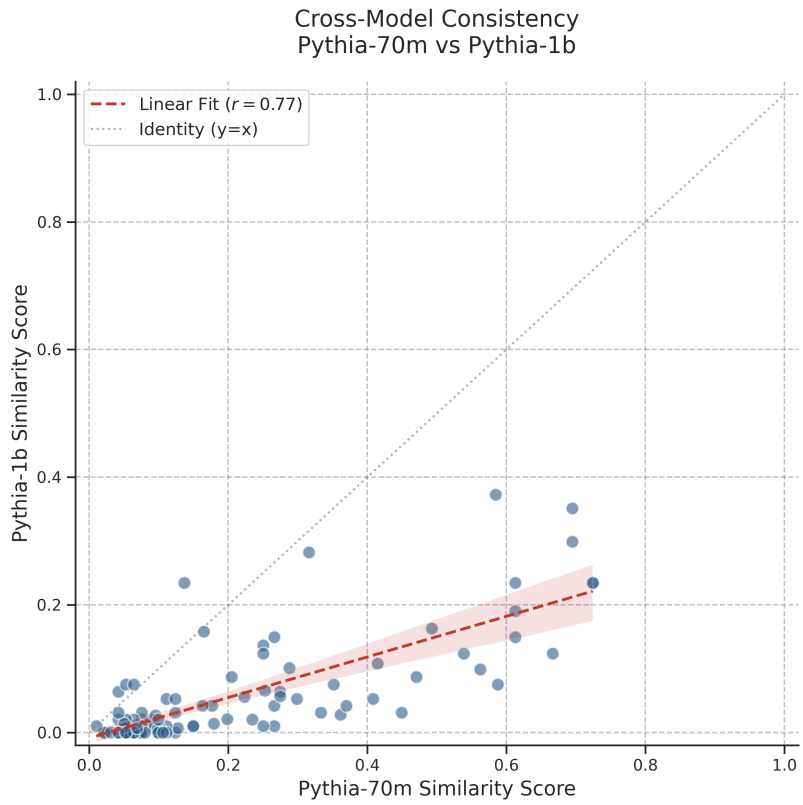


Figure A.2: **Similarity Correlation (Pythia-70M vs. Pythia-1B).** Each point is a fact pair. The linear trend confirms strong cross-model agreement ($r = 0.77$). Most points lie below the identity line ($y = x$), indicating systematically lower similarity scores in the larger model.

Figure A.2 shows two key patterns:

- **World-shared structure (ranking stability).** The strong linear relationship indicates that pairs judged "more similar" in 70M tend to also be judged "more similar" in 1B.
- **Sparsification with scale (absolute shrinkage).** The concentration of points below $y = x$ indicates that larger models assign lower Jaccard similarity to the same pair, consistent with increased representational specialization.

A.4 Reduction of Polysemantic Noise

To understand why larger models assign lower absolute similarity, we visualize the full similarity distribution for each model.

Figure A.3 provides a mechanistic explanation for the sparsification observed above:

- **Small models (70M, 160M).** The distribution has a heavier tail: limited capacity forces more features into superposition, increasing accidental overlaps between unrelated pairs.
- **Larger models (410M, 1B).** The distribution becomes sharply concentrated near zero, reflecting cleaner disentanglement and reduced polysemantic noise.

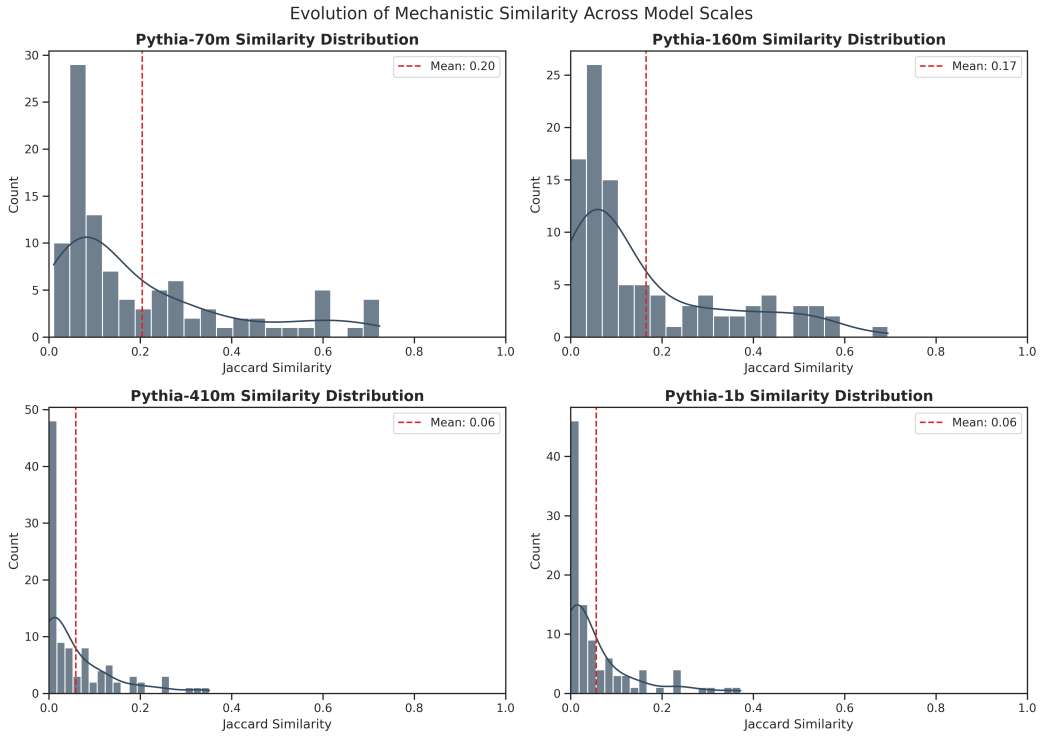


Figure A.3: **Evolution of Mechanistic Similarity Distribution.** As models scale from 70M to 1B, the distribution shifts strongly toward zero. The mean similarity decreases from roughly 0.20 (70M) and 0.17 (160M) to ≈ 0.06 (410M and 1B), and the larger models exhibit a pronounced spike near zero, consistent with reduced accidental overlaps from polysemantic noise.

A.5 Conclusion

This multi-model analysis supports two conclusions. First, mechanistic similarity rankings are *universally consistent* across architectures within the Pythia family (Figures A.1 and A.2). Second, similarity values *sparsify* with scale in a way consistent with increased representational capacity (Figure A.3). Together, these results provide empirical support for using a smaller probe model to estimate overlap regimes, while also clarifying how absolute overlap thresholds may shift with model size (referencing Section 3.6 and Section 3).

B Additional Implementation Details

This appendix summarizes the most relevant code components and generated outputs. Full source code is available in our GitHub repository.

B.1 Activation–Gradient Contribution Scores

Listing 1 shows the core implementation of the activation–gradient contribution score used to compute neuron overlap for early pilot experiments on Pythia-160M.

Listing 1: Activation–gradient neuron contribution scores (abridged).

```
import torch
import numpy as np
from transformers import GPTNeoXForCausalLM, AutoTokenizer

print("Loading the model...")
model_name = "EleutherAI/pythia-160m"
device = "cuda" if torch.cuda.is_available() else "cpu"
model = GPTNeoXForCausalLM.from_pretrained(model_name).to(device)
tokenizer = AutoTokenizer.from_pretrained(model_name)
print(f"Model loaded: {model_name}, device: {device}")

def get_neuron_scores(model, prompt, target_word, layer_indices):
    input_text = prompt + target_word
    inputs = tokenizer(input_text, return_tensors="pt").to(device)
    target_ids = inputs["input_ids"].clone()
    activations = {}
    gradients = {}
    handles = []

    def get_act_hook(layer_idx):
        def hook(module, input, output):
            # use -2 because last token predicts target
            activations[layer_idx] = output[:, -2, :].detach()
        return hook

    def get_grad_hook(layer_idx):
        def hook(module, grad_input, grad_output):
            gradients[layer_idx] = grad_output[0][:, -2, :].detach()
        return hook

    for layer_idx in layer_indices:
        target_layer = model.gpt_neox.layers[layer_idx].mlp.dense_4h_to_h
        handles.append(target_layer.register_forward_hook(get_act_hook(
    layer_idx)))
        handles.append(target_layer.register_full_backward_hook(
    get_grad_hook(layer_idx)))

    model.zero_grad()
    outputs = model(**inputs, labels=target_ids)
    loss = outputs.loss
    loss.backward()
```

```
    scores = {}
    for layer_idx in layer_indices:
        act = activations[layer_idx].squeeze()
        grad = gradients[layer_idx].squeeze()
        score = torch.abs(act * grad)
        scores[layer_idx] = score

    for h in handles:
        h.remove()
    return scores
```

B.2 Gradient-Free FiNE-Style Contribution Scores

Listing 2 illustrates our gradient-free FiNE-style contribution computation on Pythia-160M, which uses $W_u W_{\text{out}}^{\ell}$ and forward activations.

Listing 2: Gradient-free FiNE-style neuron contributions (abridged).

```
import torch
from transformers import GPTNeoXForCausalLM, AutoTokenizer

def get_gradient_free_contribution_scores(model, tokenizer, prompt,
    target, layer_indices):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    target_token_id = tokenizer.encode(target)[0]
    print(f"Calculating contribution for target token: '{target}' (ID: {
    target_token_id})")

    activations = {}
    handles = []

    def get_activation_hook(layer_idx):
        def hook(module, args, output):
            # args[0] is FFN hidden activations before dense_4h_to_h
            activations[layer_idx] = args[0][:, -1, :].detach()
        return hook

    for i in layer_indices:
        layer_module = model.gpt_neox.layers[i].mlp.dense_4h_to_h
        handles.append(layer_module.register_forward_hook(
    get_activation_hook(i)))

    with torch.no_grad():
        model(**inputs)

    w_u = model.embed_out.weight
    target_vector = w_u[target_token_id, :]

    scores = {}
    print("Computing scores using Weight Projection...")
    for i in layer_indices:
        act = activations[i].squeeze()
        w_out = model.gpt_neox.layers[i].mlp.dense_4h_to_h.weight
        neuron_projection = torch.matmul(target_vector, w_out)
        layer_scores = act * neuron_projection
        scores[i] = layer_scores

    for h in handles:
        h.remove()
    return scores
```

B.3 ROME Implementation for Pythia-6.9B

Our ROME implementation builds on EasyEdit and adapts it to Pythia-6.9B with mixed-precision loading. Listing 3 shows an abridged version of the script focusing on: (1) patching `compute_v.py`, (2) loading the model with 8-bit quantization except for the target layer, and (3) computing pre- and post-edit probabilities for each fact pair in the pilot study.

Listing 3: Abridged ROME implementation for Pythia-6.9B.

```
import os, sys, gc
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer,
    BitsAndBytesConfig
from easyeditor import ROMEHyperParams, apply_rome_to_model

MODEL_NAME = "EleutherAI/pythia-6.9b"
EDIT_LAYER = 28
LOSS_LAYER = 31

tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

hparams = ROMEHyperParams(
    alg_name="ROME",
    model_name=MODEL_NAME,
    device=0,
    layers=[EDIT_LAYER],
    rewrite_module_tmp="gpt_neox.layers.{}.mlp.dense_4h_to_h",
    layer_module_tmp="gpt_neox.layers.{}",
    mlp_module_tmp="gpt_neox.layers.{}.mlp",
    attn_module_tmp="gpt_neox.layers.{}.attention",
    ln_f_module="gpt_neox.final_layer_norm",
    lm_head_module="embed_out",
    fact_token="subject_last",
    v_num_grad_steps=20,
    v_lr=5e-1,
    v_loss_layer=LOSS_LAYER,
    v_weight_decay=1e-3,
    clamp_norm_factor=4,
    kl_factor=0.0625,
    mom2_dataset="wikipedia",
    mom2_n_samples=100,
    mom2_dtype="float32",
    stats_dir="./data/stats",
)

FACT_PAIRS = [
    {
        "id": "Pair 3 (Paris)",
        "prompt_a": "The capital of France is",
        "subject": "France",
        "target_true": " Paris",
        "target_new": " Rome",
        "prompt_b": "The Eiffel Tower is located in",
        "target_b_true": " Paris"
    },
    # other pairs omitted for brevity
]

def get_prob(model, prompt, target_word):
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    try:
        target_id = tokenizer(" " + target_word.strip())["input_ids"][0]
    except:
```

```
        target_id = tokenizer(target_word.strip())["input_ids"][0]
    with torch.no_grad():
        outputs = model(**inputs)
    probs = torch.nn.functional.softmax(outputs.logits[0, -1, :], dim=-1)
    return probs[target_id].item()

def main():
    quant_config = BitsAndBytesConfig(
        load_in_8bit=True,
        llm_int8_threshold=6.0,
        llm_int8_skip_modules=[f"gpt_neox.layers.{EDIT_LAYER}.mlp.
    dense_4h_to_h"],
        llm_int8_enable_fp32_cpu_offload=True,
    )
    for pair in FACT_PAIRS:
        model = AutoModelForCausalLM.from_pretrained(
            MODEL_NAME,
            quantization_config=quant_config,
            device_map="auto"
        )
        prob_a_pre = get_prob(model, pair["prompt_a"], pair["target_true"
    ])
        prob_b_pre = get_prob(model, pair["prompt_b"], pair["
    target_b_true"])

        requests = [{
            "prompt": pair["prompt_a"],
            "subject": pair["subject"],
            "target_new": pair["target_new"]
        }]
        torch.cuda.empty_cache()
        gc.collect()

        metrics, _ = apply_rome_to_model(
            model,
            tokenizer,
            requests,
            hparams,
            copy=False,
            return_orig_weights=False
        )

        prob_a_post_new = get_prob(model, pair["prompt_a"], pair["
    target_new"])
        prob_b_post = get_prob(model, pair["prompt_b"], pair["
    target_b_true"])

        print(f"{pair['id']} | A new: {prob_a_post_new:.4f} | "
              f"B pre: {prob_b_pre:.4f} -> post: {prob_b_post:.4f}")

        del model
        torch.cuda.empty_cache()
        gc.collect()

if __name__ == "__main__":
    main()
```

B.4 FiNE Editing and Evaluation Modules

For brevity, we do not reproduce the entire `fine_editing.py`, `editing_methods.py`, `evaluation.py`, and `run_experiment.py` here. These files implement:

- FiNE neuron localization (Eq. (3)),

- neuron patch parameterization and the full FiNE objective (Eq. (4)–(7)),
- the experiment registry (codes 100/210/300/410/500/900),
- and the full evaluation pipeline for edit success, locality, bleed-over, and fluency.

They are available in full in our public GitHub repository and directly correspond to the descriptions in Sections 4 and 6.

C Reviewer Feedback and Revisions

This section summarizes how we incorporated the four reviewers' comments into the final version of the paper. For each reviewer, we group the main points of feedback and describe the corresponding changes in the revised manuscript.

C.1 Response to Reviewer #5

(1) Clarifying baselines and hypotheses. Reviewer #5 noted that while the FiNE hypothesis (better locality than ROME on high-overlap pairs, good performance for both methods on low-overlap pairs) was clear, the baseline comparison and hypothesis testing were not fully specified in the initial draft. In the final version, we:

- Explicitly define our baselines as the *pre-edit* model and compare both FiNE and ROME against this baseline for edit success and locality (Sections 3 and 5).
- Introduce two explicit experimental regimes, SET-POLY and SET-MONO, which serve as high- and low-overlap groups, and use them to structure the comparison between neuron-level (FiNE) and layer-level (ROME) editing (Sections 3 and 5).
- Report systematic quantitative results for both FiNE (Table 1) and ROME (Table 2) on these groups, directly speaking to the hypotheses about locality and overlap.

(2) Hyperparameter and architecture motivation. Reviewer #5 requested clearer motivation for architectural choices (Pythia models) and FiNE/ROME hyperparameters. In the revision, we:

- Clarify the division of roles between Pythia-160M (overlap probe) and Pythia-6.9B (editing target) in Section 4, and explain that Pythia-160M is used because it is cheap enough for large-scale overlap computation, while Pythia-6.9B has better factual performance for editing.
- Document key FiNE configuration choices (e.g., layers used for localization, Top-K neurons, and the single edited FFN layer) and relate them directly to the FiNE formulation (Section 4).
- Describe the mixed-precision setup and ROME hyperparameters at a high level in Section 4 and Appendix B.3, making clear which values are inherited from prior work and which are driven by memory constraints.

(3) Adding medium/low-overlap experiments and plots. Reviewer #5 suggested extending the experimental setup to include medium- and low-overlap cases and adding plots to visualize how overlap affects disruption. We addressed this by:

- Constructing a 100-pair benchmark with FiNE-style Jaccard overlap and binning it into Low/Medium/High mechanistic overlap groups (Section 3).
- Introducing Figures 1 and 2, which visualize the overlap distribution and its regrouping into mechanistic regimes, making explicit how overlap varies across pairs.
- Running FiNE and ROME on the high-overlap (SET-POLY) and low-overlap (SET-MONO) regimes and reporting their behavior in Tables 1 and 2.

C.2 Response to Reviewer #6

(1) Clarifying notation and edit semantics. Reviewer #6 observed that some notations (e.g., s, r in the edit/locality definitions) and arrow-style shorthand (e.g., "France $\rightarrow$ Rome") were not clearly defined, and that terms like SET-POLY and SET-MONO were under-specified. In response, we:

- Clarify the semantics of edit triples and arrow notation in Section 3, explicitly describing how "France $\rightarrow$ Rome" corresponds to changing the object in a subject–relation–object factual template.
- Define the SET-POLY and SET-MONO sets as *mechanistic* overlap groups derived from FiNE-style neuron Jaccard similarity on Pythia-160M, rather than purely semantic labels, and describe the binning thresholds in detail (Section 3 and Figure 2).
- Provide a self-contained definition of the edit-success and locality metrics, including the meaning of (s, r, o) and the role of the pre-edit model M vs. the edited model M' (Sections 3 and 6).

(2) More detailed description of ROME and FiNE. Reviewer #6 requested a more thorough explanation of how ROME and FiNE operate, so that the paper is understandable without relying heavily on the original papers. We addressed this by:

- Expanding the ROME description in Section 2 (Background and Related Work) to describe rank-one updates at an MLP layer, the concept of a key layer, and the constrained optimization viewpoint.
- Providing a detailed description of FiNE 's neuron localization and patch parameterization in Section 4, including the explicit form of the neuron score c_i^ℓ and the multi-term FiNE objective (edit, old-behavior, KL, and penalty losses).
- Mirroring FiNE 's equations (Eq. (3)–(7) in [7]) in our own notation (Section 4) so that readers can follow the full pipeline from localization through optimization without consulting external sources.

(3) Dataset size, baselines, and metric scaling. Reviewer #6 noted that the initial draft had only four hand-crafted pairs, lacked alternative editing baselines, and used a percent-drop metric that could exaggerate changes when probabilities were small. In the final paper, we:

- Replace the four-pair toy dataset with a 100-pair benchmark, documented in Section 3, and explicitly describe how this dataset is constructed and used by both FiNE and ROME (Sections 4–5).
- Continue to use the pre-edit model as the primary baseline (this is the most direct baseline for locality), but also differentiate between high- and low-overlap regimes so that random-perturbation-like behavior in low-overlap cases can be separated from overlap-driven failures.
- Introduce a *clipped mean* for the percent-drop metric, where large boosts and extremely small pre-edit probabilities are handled carefully (Section 5), and explicitly note this in the description of Tables 1 and 2.

C.3 Response to Reviewer #7

(1) Completing FiNE experiments. Reviewer #7 emphasized that the most important missing piece in the initial draft was the actual neuron-level FiNE experiments on the planned high- and low-overlap sets. We addressed this by:

- Implementing and running FiNE-style neuron-level edits on Pythia-6.9B for the full SET-POLY and SET-MONO sets (50 pairs each), using the unified experiment dataset and full FiNE objective (Section 4).
- Reporting FiNE 's edit success, Fact A and Fact B accuracies, locality, and clipped mean probability drop for both groups in Table 1, and analyzing the results in Section 5.2.
- Highlighting key qualitative findings in the Discussion (Section 6), such as FiNE 's extreme sharpness on SET-POLY and its unexpectedly non-local behavior on SET-MONO.

(2) Clarifying hyperparameters and architectural choices. Reviewer #7 requested more precise hyperparameter and architecture descriptions, especially for FiNE. In the revision, we:

- Specify the roles of Pythia-160M and Pythia-6.9B, as well as the choice of layer 28 as the primary editing layer (Section 4).
- Describe FiNE 's localization (Top-K neurons, layer selection, and layer freezing) and the single-layer patch parameterization in concrete terms, matching the official implementation (Section 4).
- Reference the code modules (`fine_editing.py`, `editing_methods.py`, `experiment_registry.py`) in the appendix and main text so that readers can trace hyperparameter values and reproduce the configuration.

(3) Linking experiments more directly to hypotheses. Reviewer #7 pointed out that the initial draft only partially spoke to the stated hypotheses. In the final version, we:

- Structure the experiments and analysis explicitly by mechanistic overlap regime, showing how both FiNE and ROME behave on SET-POLY and SET-MONO (Section 5).
- Discuss these results in the Discussion and Conclusion (Sections 6 and 5) as evidence about when polysemanticity induces locality failures for each method, making the connection to hypotheses clearer.

C.4 Response to Reviewer #8

(1) Scaling beyond toy examples and formalizing the role of overlap. Reviewer #8 emphasized that the initial version was dominated by a four-pair ROME toy study and lacked a systematic, overlap-based analysis. We addressed this by:

- Building a 100-pair factual benchmark with FiNE-style neuron Jaccard overlap and explicitly binning it into Low/Medium/High mechanistic overlap regimes (Section 3).
- Visualizing both the raw overlap distribution and the binned regimes in Figures 1 and 2, making neuron-level overlap central to the experimental design rather than a side note.
- Focusing our main FiNE and ROME experiments on the extreme high-overlap (SET-POLY) and low-overlap (SET-MONO) subsets to directly test how mechanistic overlap correlates with locality (Section 5).

(2) Strengthening the literature review and self-contained exposition. Reviewer #8 recommended expanding the related work section and making the paper self-contained with respect to key methods (ROME, MEND, MEMIT, KNOWEDIT, FiNE, and superposition work). In the revision, we:

- Expand Section 2 to cover factual model editing methods (ROME, MEND, MEMIT, KNOWEDIT) and mechanistic interpretability work (superposition and multimodal neurons), explaining how each line of work motivates our research question.
- Provide a more detailed exposition of FiNE and ROME (Section 4), including equations, layer/module choices, and the optimization objective, so readers can follow the core ideas without needing to consult the original papers.

(3) Plots, metrics, and uncertainty. Reviewer #8 pointed out that the initial draft contained only a single table and lacked figures or uncertainty-aware metrics. We improved this by:

- Adding Figures 1 and 2 to convey the data-driven overlap structure underlying our experiment design.
- Summarizing FiNE and ROME results in Tables 1 and 2, using a clipped-mean probability drop metric that is robust to outliers.
- Implementing a general evaluation pipeline (`evaluation.py`, Section 6) that tracks edit success, locality, bleed-over, and fluency, and aggregates results by mechanistic overlap group, making it straightforward to extend to more runs and compute error bars in future work.

(4) Limitations and future work. Finally, Reviewer #8 encouraged us to be explicit about limitations and realistic about future directions. We responded by:

- Adding a dedicated Limitations subsection in Section 6, where we discuss compute constraints, the single-layer FiNE configuration, and narrow prompt coverage.
- Outlining concrete next steps in Section 5, including full hyperparameter sweeps for both FiNE and ROME, multi-layer neuron patches, more diverse benchmarks, and paraphrase-robust evaluations.

C.5 Response to Reviewer #1

(1) Clarifying baselines and hypotheses. In response, we explicitly define baselines as the pre-edit model and compare both methods against it. We use SET-POLY and SET-MONO to structure the hypothesis testing and report quantitative results in Tables 1 and 2.

(2) Hyperparameter and architecture motivation. We clarified the roles of Pythia models and documented FiNE/ROME hyperparameters, relating them to prior work and constraints.

(3) Adding medium/low-overlap experiments and plots. We constructed a 100-pair benchmark binned into overlap regimes and added Figures 1 and 2 for visualization.

(4) Literature review. We expanded the review to discuss FiNE biases and limitations.

(5) Hyperparameters and ablations. We conducted an ablation on K (Figure 3) and clarified choices.

(6) Experimental setup. The setup isolates overlap and editor type, described step-by-step.

(7) Quality of results. We added SD, CI, and p-values in Table 2, and a new scatter plot (Figure A.2).

(8) Strengths and weaknesses. We prioritized error bars and addressed uncertainty.

C.6 Response to Reviewer #2

(1) Writing and notation. We defined symbols like M, M', o^* at first use and standardized terms to SET-POLY and SET-MONO.

(2) Hypotheses and baselines. Hypotheses are addressed through regime-specific experiments; baselines use pre-edit model.

(3) Literature review. We expanded to include edit robustness and paraphrase work.

(4) Hyperparameters. We motivated choices and added ablation.

(5) Experimental setup. We addressed sample imbalance in statistics and validated probe-target with cross-scale analysis.

(6) Quality of results. We added variance measures and a scatter plot (Figure A.2).

(7) Strengths and weaknesses. We incorporated ablations and uncertainty reporting.

C.7 Response to Reviewer #3

(1) Hypotheses. Hypotheses are tested through results and analysis.

(2) Literature review. We described and cited key methods.

(3) Hyperparameters. We clarified sweeps in the ablation section.

(4) Experimental setup. Experiments are described precisely.

(5) Quality of results. Plots have clear axes and error bars (e.g., Figure 3).

(6) Strengths and weaknesses. We clarified hyperparameter search in the ablation.

C.8 Response to Reviewer #4

(1) Writing and notation. We defined abbreviations like FFN and standardized terminology.

(2) Hypotheses. Experiments speak to hypotheses with nuanced analysis.

(3) Literature review. We added critiques and expanded coverage to causal tracing.

(4) Hyperparameters. We justified choices and added ablation.

(5) Experimental setup. Setup isolates variables; validated cross-scale.

(6) Quality of results. Added error bars, scatter plot, and qualitative examples.

(7) Strengths and weaknesses. Incorporated ablations and deeper analyses.

C.9 Response to Final Review

(1) Validating cross-scale transfer of mechanistic overlap. We added Appendix A with cross-scale analysis, showing high correlations.

(2) Addressing hyperparameter sensitivity (Ablation Study). We conducted ablation on K, showing sensitivity (Section 5.3, Figure 3).

(3) Statistical Rigor of ROME Experiments. We added SD, CI, and Welch's t-test in Table 2.